CiT Backend Engineering Track

Week 18: Caching & Performance

Measuring Speed, Fixing Bottlenecks, Database Indexes, Redis & Spring Cache

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 2 — Specialisation (Weeks 9–20)
Week	Week 18
Instructor	Jim & Anderson
Duration	6 months | 24 weeks
Tech Stack	Java 25, Spring Boot 4, Redis, PostgreSQL, Docker

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Thank You for every hour, every skill, and every resource You have placed in our hands. We confess that we do not always use them well. Teach us to be wise stewards — to keep close what is truly valuable, and not to waste the time, energy, and gifts You have given us.

As Proverbs 21:20 reminds us, "There is desirable treasure, and oil in the dwelling of the wise, but a foolish man squanders it." Lord, make us wise in how we spend our days, careful in how we build, and generous in how we share.

Let our work and our code serve people without wasting their time or their money. Give us patience to measure before we act, and humility to learn from what we find. May all we build bring honour to Your name.

In Jesus' name we pray,
Amen.

Scripture for the Week

"There is desirable treasure, and oil in the dwelling of the wise, but a foolish man squanders it."

— Proverbs 21:20 (NKJV)

Table of Contents

1. Welcome to Week 18
2. Measure Before You Optimise
3. Common Bottlenecks
4. Database Indexes
5. What Is Caching?
6. Redis Fundamentals
7. Spring Cache with Redis
8. Putting It Together — Speeding Up the Library API
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 18

Last week you completed the Messaging module: JSON events, retries with back-off, dead-letter topics, idempotent consumers, the transactional outbox, and an event-driven notification service. Your Library platform now does slow work in the background.

This week begins the Caching, Performance & DevOps Basics module (Weeks 18–20): speed this week, Docker in Week 19, and automated testing with CI/CD in Week 20.

Speed is not a luxury. Many of our users are on mobile data, and every unnecessary second costs them patience and bundles. The first step is not to guess, but to measure.

What to Expect in Week 18

By the end of this session, you will be able to:

- Measure response times and explain p50, p95, and p99 in plain English
- Recognise the most common bottlenecks, including N+1 queries and missing indexes
- Read an `EXPLAIN ANALYZE` plan and add an index in PostgreSQL and in JPA
- Explain caching: hits, misses, TTL, invalidation, and what not to cache
- Run Redis locally and use its core commands
- Cache with `@Cacheable` and keep the cache correct with `@CacheEvict`
- Speed up the Library API step by step, proving each change with numbers

2. Measure Before You Optimise

When an app feels slow, resist changing code at once. Most guesses about "what is slow" are wrong, and a change made without measuring cannot be proven to help.

2.1 What Does "Fast" Mean?

Response time (also called latency) is how long one request takes, from arrival to reply, measured in milliseconds (ms) — 1,000 ms is one second.

2.2 Averages Hide the Slow Requests

Imagine 100 requests: 95 take 80 ms, but 5 take 4 seconds. The average, 276 ms, describes nobody. So engineers use percentiles:

Percentile	Plain English
p50 (median)	Half of requests are faster than this — the typical experience
p95	95 in every 100 requests are faster than this — the slow-ish end
p99	99 in every 100 are faster than this — the unlucky 1 in 100

> **Think of Boda-Boda Trips**
>
> If your boda ride usually takes 15 minutes but one day in twenty it takes an hour, a good average is no comfort on that day. The p95 tells you about that bad day — and users remember the slow requests.

2.3 Tools for Measuring

Actuator (Week 12). Expose the metrics endpoint, then ask for the request timer of one endpoint:

```
management:
  endpoints.web.exposure.include: health,metrics
  metrics.distribution.percentiles.http.server.requests: 0.5, 0.95, 0.99

GET /actuator/metrics/http.server.requests?tag=uri:/api/v1/books/{id}
```

The reply shows COUNT, TOTAL_TIME, and MAX (the slowest). The percentiles line asks Spring to record p50, p95, and p99 too. Keep Actuator restricted (Week 12).

A log line. To time one suspicious method, measure it directly:

```
long start = System.nanoTime();
List<LoanResponse> loans = loanService.findOverdue();
long ms = (System.nanoTime() - start) / 1_000_000;
log.info("findOverdue returned {} loans in {} ms", loans.size(), ms);
```

SQL logging and Postman. Set logging.level.org.hibernate.SQL: debug to see every query. Postman shows each response time, and its Collection Runner repeats requests.

Measure, Change One Thing, Measure Again

Record a baseline, change one thing only, and measure again under the same conditions. Change three things at once and you cannot tell which one helped. Ignore the first request after start-up — the JVM is still warming up.

Exercise 1: Find Your Slowest Endpoint

1. Expose Actuator metrics in your Library API (locally only)
2. Call each GET endpoint 20 times with the Collection Runner; note COUNT, TOTAL_TIME, and MAX
3. Write down your baseline table — you will need it again in Section 8

3. Common Bottlenecks

A bottleneck is the narrow neck of a bottle: water only pours as fast as the neck allows. In an API, one slow step limits the whole request, and the same few causes appear again and again.

3.1 The N+1 Query Problem (Week 11)

Listing 50 loans runs one query for the loans, then one more per loan for its member — 51 queries where one would do. It hides in development and appears in production.

```
@Query("SELECT l FROM Loan l JOIN FETCH l.member JOIN FETCH l.book " +
       "WHERE l.status = :status")
List<Loan> findWithMemberAndBook(@Param("status") LoanStatus status);
```

On the XENO platform OSIV is off (spring.jpa.open-in-view: false), so accidental lazy loads fail loudly.

3.2 Other Usual Suspects

- **Missing indexes** — the database reads every row to find a few (Section 4).
- **Fetching too much** — all 20,000 books with no pagination, or whole entities when three fields are needed. Paginate (Week 11) and use small DTOs.
- **Slow remote calls inside the request** — an SMS gateway called while the user waits (Week 16). Set timeouts; move such calls out.
- **Work that could be asynchronous** — receipts and reminders can be events or scheduled jobs (Week 17).

Symptom	Likely Cause	Fix
Many near-identical SQL lines per request	N+1 queries	JOIN FETCH or a DTO projection
One query slow, gets worse as data grows	Missing index	Add an index on the filtered or joined column
Large responses, slow on mobile data	Fetching too much	Pagination and smaller DTOs
Fast sometimes, very slow other times	Slow remote call	Timeouts; move it to an event
Same unchanged data read constantly	Repeated work	Cache it (Sections 5–7)

Exercise 2: Diagnose the Symptoms

Name the likely cause and a first fix:

4. "The members list was fine last month, but now it takes 6 seconds."
5. "The SQL log shows 101 queries when I open the loans page."
6. "Borrowing a book sometimes hangs for 10 seconds while the SMS sends."
7. "The books endpoint returns 4 MB of JSON to the mobile app."

4. Database Indexes

The single most effective speed-up in most APIs is a well-chosen index. It is also one of the most often forgotten.

4.1 The Index at the Back of a Book

To find "photosynthesis" in a 600-page biology textbook, you turn to the index at the back, not page one. A database index is the same: a sorted structure pointing straight to the matching rows. Without one, PostgreSQL reads every row — a sequential scan.

4.2 Ask PostgreSQL How It Runs a Query

Put EXPLAIN ANALYZE in front of a query and PostgreSQL runs it and reports its plan and timing (illustrative output below — your numbers will differ):

```
EXPLAIN ANALYZE SELECT * FROM loan WHERE member_id = 42;

Seq Scan on loan  (actual time=0.031..14.902 rows=12 loops=1)
  Filter: (member_id = 42)
  Rows Removed by Filter: 99988
Execution Time: 14.950 ms
```

PostgreSQL read 100,000 rows to find 12. Now add an index and run the same query again:

```
CREATE INDEX idx_loan_member_id ON loan (member_id);

Index Scan using idx_loan_member_id on loan  (actual time=0.020..0.041 rows=12 loops=1)
  Index Cond: (member_id = 42)
Execution Time: 0.070 ms
```

Same answer, a fraction of the work. You may also see a "Bitmap Index Scan" — that uses the index too.

4.3 Declaring an Index in JPA

You can also declare the index on the entity, so it is created whenever Hibernate generates the schema:

```
@Entity
```

```
@Table(name = "loan",
    indexes = @Index(name = "idx_loan_member_id", columnList = "member_id"))
public class Loan { ... }
```

The annotation only takes effect when Hibernate creates the tables; in a real team, make sure the index also exists in every environment's schema scripts.

4.4 Which Columns Need an Index?

PostgreSQL automatically indexes primary keys and unique columns. It does NOT automatically index foreign-key columns such as loan.member_id or loan.book_id — you must add those yourself. Good candidates are columns you filter on (WHERE), join on (JOIN), or sort by (ORDER BY) in frequent queries.

Indexes Are Not Free

Every index takes disk space and slows every INSERT, UPDATE, and DELETE a little. Index the columns your real queries use, not every column "just in case". On a tiny table PostgreSQL may still choose a sequential scan — that is not a bug.

Exercise 3: Prove an Index Works

8. Insert 100,000 test loans locally (generate_series helps)
9. Run EXPLAIN ANALYZE on a query filtering by book_id and record the plan and time
10. Create idx_loan_book_id, run it again, and compare
11. List your foreign-key columns and decide which need an index

5. What Is Caching?

Indexes make a query faster. Caching avoids running it at all.

5.1 Stock at the Counter

The Shopkeeper's Counter

A shopkeeper in Nakasero keeps the fast-moving items — airtime, sugar, soap — on the counter, within arm's reach. Everything else stays in the store-room at the back. Most customers are served in seconds; only the occasional unusual request means a trip to the back. A cache is the counter: a copy of frequently requested answers, kept close by.

In our API, the store-room is PostgreSQL. We check the cache first; if the answer is there we return it, if not we ask the database and keep a copy for next time. This pattern is called cache-aside.

5.2 The Vocabulary

Term	Plain English
Cache hit	The answer was in the cache — fast
Cache miss	It was not — fetch it from the database and store a copy
TTL	Time To Live: how long a copy is kept before it expires automatically
Invalidation	Removing (evicting) a copy because the real data has changed
Stale data	A cached copy that no longer matches the database

Every cache trades freshness for speed: a title cached for ten minutes may be ten minutes out of date — unless you evict it when the book changes. An old joke says the two hard things in computing are cache invalidation and naming things.

5.3 What to Cache — and What Not To

Good to Cache	Do Not Cache
The book catalogue — read constantly, changes rarely	Account or wallet balances that must be exactly right

Book details and categories	Per-user data stored under a shared key
Reference lists: districts, currencies	Permissions and roles
Expensive reports that can be minutes old	Data that changes on almost every request

A Wrong Key Leaks Data

Cache "my loans" under the key "my-loans" instead of "my-loans:<member id>", and the second member receives the first member's loans — a data leak. Data that depends on who is asking must include the user in its key, or not be cached. Never cache permissions: a revoked role must stop working at once.

Exercise 4: Cache It or Not?

For a school fees system, cache each item or not? Suggest a TTL where you would:

12. The list of classes and their fees for this term
13. A parent's outstanding balance before paying via mobile money
14. The school's public contact details
15. The bursar's end-of-day collections report

6. Redis Fundamentals

Redis is an in-memory key-value store: it holds data in RAM, where reading is extremely fast — like a shared Java Map that many applications reach over the network.

6.1 Run Redis Locally

As with Kafka in Week 16, Docker is the quickest way (you will study Docker properly next week):

```
docker run -d --name redis -p 6379:6379 redis
docker exec -it redis redis-cli
```

The second command opens redis-cli, Redis's command-line client. Type PING and Redis answers PONG.

6.2 The Core Commands

```
SET book:42 "Things Fall Apart"     # store a value under a key
GET book:42                         # "Things Fall Apart"
EXPIRE book:42 600                  # delete it automatically after 600 seconds
TTL book:42                         # seconds left; -1 = no expiry, -2 = no such key
DEL book:42                         # remove it now
INCR visits:home                    # add 1 atomically and return the new count
```

6.3 Data Types in Brief

Type	Holds — and a Typical Use
String	Text, numbers, or serialised objects — cached responses, counters
Hash	Field-value pairs under one key — a small object such as a session
Set	Unique values — the ids of members online now
Sorted Set	Unique values with scores, kept in order — a leaderboard

6.4 Redis on the XENO Platform

On the XENO platform, Redis is used for caching, session storage, and general key-value needs. A classic use is rate limiting (Week 14): say, 10 login attempts per minute.

```
INCR rate:login:0772123456        # 1 on the first attempt
EXPIRE rate:login:0772123456 60   # set only when the count is 1
# if INCR ever returns more than 10 → reply 429 Too Many Requests
```

INCR is atomic, so two simultaneous requests never read the same count. After a minute the key expires and counting restarts.

Redis Is the Counter, Not the Store-Room

Memory is limited and Redis data can be lost on restart. PostgreSQL remains your source of truth: an empty cache should make things slower, never wrong.

Exercise 5: Redis by Hand

16. Start Redis in Docker and open redis-cli
17. Store a book title with a 30-second expiry; watch TTL count down, then GET it after it expires
18. Simulate the login rate limiter: run INCR and EXPIRE on a key and call INCR eleven times
19. Explain why rate limiting with INCR is safe when many requests arrive together

7. Spring Cache with Redis

Rather than hand-writing cache-aside code, use Spring's cache abstraction: add an annotation, and a proxy checks the cache, calls your method on a miss, and stores the result.

7.1 Dependencies and Configuration

On start.spring.io (or in an existing project's build file), add Spring Data Redis and Spring Cache Abstraction:

```
implementation 'org.springframework.boot:spring-boot-starter-data-redis'
implementation 'org.springframework.boot:spring-boot-starter-cache'
```

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
  cache:
    type: redis
    redis:
      time-to-live: 10m         # every cached entry expires after 10 minutes
```

Then switch caching on once, in a configuration class:

```
@Configuration
@EnableCaching
public class CacheConfig { }
```

7.2 @Cacheable — Remember the Answer

```
public record BookResponse(Long id, String title, String author,
                           String isbn, int availableCopies) implements Serializable { }

@Cacheable(cacheNames = "books", key = "#id")
public BookResponse findById(Long id) {
    log.info("Cache miss — loading book {} from the database", id);
    return bookMapper.toResponse(bookService.getById(id));
}
```

The first call for book 42 logs "Cache miss" and stores the answer under the Redis key books::42. Later calls return straight from Redis; the method body does not run. #id is the id parameter.

Cache DTO Records, Never Entities

Cache facade methods that return DTOs, not entities, which carry lazy-loading proxies and internal fields. The default Redis cache uses Java serialisation, so every cached record must implement Serializable — forget it and the first cache write fails.

7.3 Keeping the Cache Correct

```
@Caching(evict = {
   @CacheEvict(cacheNames = "books", key = "#id"),
   @CacheEvict(cacheNames = "bookPages", allEntries = true)
})
public BookResponse update(Long id, BookRequest request) { ... }
```

Annotation	Does	Typical Use
@Cacheable	Return the cached value, or run and store	Reads: find a book by id
@CacheEvict	Remove one entry, or all with allEntries = true	Updates and deletes
@CachePut	Always run, then store the new result	Refresh an entry after an update
@Caching	Group several cache annotations on one method	Evict from two caches at once

The Self-Invocation Trap

Caching works through a proxy, exactly like @Transactional. If a method calls a @Cacheable method in the same class, the call never passes through the proxy and the cache is silently skipped. Call cached methods from another bean, such as the controller.

Exercise 6: Cache Book Details

20. Add the two dependencies and the configuration, and annotate a config class with @EnableCaching

21. Make BookResponse Serializable and add @Cacheable to the facade's findById
22. Call GET /api/v1/books/1 three times; confirm "Cache miss" is logged only once
23. In redis-cli, run TTL books::1, then update the book and check the key has gone

8. Putting It Together — Speeding Up the Library API

Apply the week to the Library API in the right order: measure, fix, measure again, and only then cache.

8.1 Step 1 — Measure the Baseline

Use your Exercise 1 table. Suppose the slow endpoints are the active-loans list, a member's loans, and catalogue page 1.

8.2 Step 2 — Fix the N+1 Query

The SQL log shows extra queries per loan. Switch to the JOIN FETCH query from Section 3.1, then measure.

8.3 Step 3 — Add the Missing Indexes

```
CREATE INDEX idx_loan_member_id ON loan (member_id);
CREATE INDEX idx_loan_book_id   ON loan (book_id);
CREATE INDEX idx_loan_status    ON loan (status);
```

Confirm each one with EXPLAIN ANALYZE, then measure again.

8.4 Step 4 — Cache Book Details and the First Catalogue Page

Book details are cached since Exercise 6. Almost everyone sees catalogue page 1, so cache only that page:

```
public record BookPageResponse(List<BookResponse> items, int page,
                               long totalItems) implements Serializable { }

@Cacheable(cacheNames = "bookPages", key = "#page + '-' + #size",
        condition = "#page == 1")
public BookPageResponse listBooks(int page, int size) { ... }
```

8.5 Step 5 — Evict on Every Change

Adding, updating, or deleting a book must evict it from "books" and clear "bookPages" (Section 7.3). Borrowing or returning changes availableCopies, so loan methods must evict that book too. Check: change a book, fetch it — the new value must appear at once.

8.6 Step 6 — Measure Again

Endpoint (p95)	Before	After
GET /loans?status=ACTIVE	1,850 ms (N+1)	120 ms (JOIN FETCH + index)
GET /members/{id}/loans	640 ms (no index)	35 ms (index)
GET /books/{id}	45 ms	4 ms on a cache hit
GET /books?page=1	310 ms	6 ms on a cache hit

Illustrative figures — yours will differ. What matters is that every improvement is measured, not assumed.

What Just Happened

The biggest wins came from the database fixes, not the cache. Caching then made good reads faster and took load off PostgreSQL. Fix queries first, cache second: a cache in front of an N+1 query only hides it until the entry expires.

Exercise 7: Speed Up Your Library API

24. Record a baseline for your three slowest endpoints
25. Fix any N+1 queries and add the indexes your queries need; measure after each change
26. Cache book details and catalogue page 1, with eviction on every change to a book
27. Produce your before/after table and say which change helped most

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Measure before you optimise; watch p95 and p99, not just the average
- Actuator metrics, timing log lines, SQL logging, and Postman timings as measuring tools
- Common bottlenecks: N+1 queries, missing indexes, fetching too much, slow remote calls, avoidable synchronous work
- Indexes turn sequential scans into index scans; PostgreSQL does not index foreign keys for you
- Caching keeps answers close by; hits, misses, TTL, invalidation, and what never to cache
- Redis core commands, its data types, and rate limiting with INCR and EXPIRE
- Spring Cache with @Cacheable, @CacheEvict, and @CachePut; cache Serializable DTO records; beware self-invocation

9.2 Homework

28. Complete Exercise 7 and push your faster Library API, with its before/after table in the README, to GitHub
29. Add a rate limit of 10 requests per minute per member to one endpoint using Redis INCR and EXPIRE
30. List every cache with its key, TTL, and the actions that evict it
31. Preview Week 19: Read about the difference between a Docker image and a container, and what a Dockerfile is. Come with one question ready.

Coming Next — Week 19: Docker & Docker Compose

Jim & Anderson will finally explain the docker run commands you have been typing since Keycloak. You will write a Dockerfile for your Spring Boot API, build and run it as a container, and use Docker Compose to start the whole Library platform — API, PostgreSQL, Redis, and Kafka — with a single command.

Closing Reflection

"See then that you walk circumspectly, not as fools but as wise, redeeming the time, because the days are evil."

— Ephesians 5:15–16 (NKJV)

To walk circumspectly is to look carefully at where you are going. This week you learned to look before you change anything: to measure, then act. Every second

you save a user is a small act of care for someone's time. Use your own time the same way — wisely, carefully, and for what truly matters.

Glossary of Terms

Term	Definition
Bottleneck	The slowest step in a process, which limits the speed of the whole
Cache	A fast store holding copies of frequently requested answers
Cache Hit / Miss	Finding the answer in the cache, or not finding it and fetching it from the source
Cache Invalidation	Removing a cached copy because the real data has changed
EXPLAIN ANALYZE	A PostgreSQL command that runs a query and reports its plan and timing
Index	A sorted structure that lets the database find matching rows without reading them all
N+1 Query Problem	One query for a list, then one extra query for every item in it
Percentile (p95)	The value below which that percentage of measurements fall
Rate Limiting	Restricting how many requests a client may make in a period of time
Redis	An in-memory key-value store used for caching, sessions, and counters
Sequential Scan	Reading every row in a table to find the matches
Stale Data	A cached copy that no longer matches the source of truth
TTL	Time To Live: how long a cached entry is kept before it expires